实验 8 基于非阻塞套接字的通信程序设计

实验目的:

1.掌握基于非阻塞套接字的通信程序设计。

实验环境:

1.联网并运行 Linux 操作系统的 PC 机

实验内容和要求:

编写一个服务器端程序和客户端程序。要求如下：

（1） 服务器端程序与客户程序的 TCP 连接建立后首先向客户端发送一条内容为“Connect succeed. Please send a message to me.”，

（2） 然后等待接收客户端发送来的一条消息，收到后显示该信息并关闭连接。

要求服务器端套接字使用非阻塞模式，允许同时有多个客户接入。客户端程序在与服务器的连接建立成功后接收并显示从服务器收到的信息，然后从键盘接收一行信息发送给服务器。

实验原理

阻塞是指一个进程/线程执行了一个函数或系统调用，该函数或系统调用由于某种原因不能立即完成，因而不能返回调用它的进程/线程，导致进程/线程处于等待状态。比如，C 程序在调用 scanf()、getc()、gets()等 I/O 函数接收用户从键盘上输入的数据时，在用户输入完成之前程序一直处于等待状态而不能向前推进，直到用户完成数据输入后，程序才会继续推进。这种函数的调用方式即为阻塞方式。

非阻塞则是指一个进程/线程调用一个函数或系统调用后就立即返回，而不需要等待被调用函数完成。

所以，阻塞和非阻塞与函数的调用有关。它们的区别是调用者要不要等待被调用函数返回结果。

使用 fcntl()函数来把套接字函数设置为非阻塞工作模式。以下代码片段用于把套接字 fd 设置为非阻塞模式：

```
int flags;
...
if((flags = fcntl(fd, F_GETFL,0)) <0)
   //出错处理
```

```
flags |= O_NONBLOCK;
if(fcntl (fd, F_SETFL,flags) < 0)
  //出错处理
```

与阻塞和非阻塞概念容易混淆的还有一对概念：同步和异步。

同步操作：指进程/线程启动一个过程后，需要自己去查询，以获取被调用过程的执行结果。

异步操作：指进程/线程启动一个过程后，可以去处理其它任务。当被调用过程完成任务后，其会主动向调用者发送消息。

即：同步和异步的区别在于是否需要调用者自己去检测被调用者工作是否已完成。

套接字即可以工作在阻塞模式也可以工作在非阻塞模式。

工作于阻塞模式的套接字调用：线程或进程调用套接字函数时，如果不满足执行条件，套接字函数将被阻塞，直到相应的条件满足时才执行成功并返回。比如调用 recv()函数读取网络缓冲区中的数据，如果没有数据到达，将一直阻塞在 recv()这个函数调用上，直到读到一些数据，此函数才返回。

工作于非阻塞模式的套接字调用：执行套接字函数时，不管套接字是否满足执行条件，执行是否成功都立即返回，该函数所在的线程继续运行。比如调用 recv()函数读取网络缓冲区中数据，不管是否读到数据都立即返回，而不会一直挂在此函数调用上。如果缓冲区中没有数据则返回错误：EWOULDBLOCK。

当套接字工作非阻塞模式下时：

（1）调用 accept()函数：不管是否有客户端的连接请求到达都会立即返回。若无客户连接请求到达，accept()函数返回值小于 0，错误码被设置为 EWOULDBLOCK。

（2）调用 recv()/recvfrom()函数：不管缓冲区中是否有数据都会立即返回。若缓冲区中无数据，则 recv()/recvfrom()函数的返回值小于 0，错误码被设置为：EWOULDBLOCK。

（3）调用 send()/sendto()：不管能否把数据发送出去都会立即返回，若不能发送数据，函数的返回值会小于 0，错误码被设置为：EWOULDBLOCK。

（4）调用 connect()：无论是否马上建立起连接都会立即返回，若没有建立连接就返回，会把错误码设置为：EINPROGRESS。

所以在非阻塞模式下调用套接字接口函数，若函数的返回值表示调用没有成功，这并不意味着调用出现错误，还需要检查错误代码，若错误代码是 EWOULDBLOCK 或者 EINPROGRESS，这表明函数的调用并没有出错，需要一段时间后再重新调用上述套接字函数来完成 I/O 操作。

套接字在非阻塞模式下，进行 I/O 操作的关键在于确定套接字何时可读/可写，也就是判断网络事件何时发生。一种编程方式是通过反复调用函数并检查返回值，直到调用成功为止。这实际上就是同步非阻塞工作方式。比如以下代码片段即是在非阻塞套接字 s 上接收数据：

```
while(true)
{
  if(recv(s, buf,sizeof(buf),0)<0)
  {
      if(errno != EWOULDBLOCK)
```

```
        {
            //接收消息失败退出
        }
    } else {
        //接收成功后对收到数据进行处理
    }
    ……
}
```

程序调用了 recv 函数后马上返回，如果此时缓冲区中并没有待处理的数据，那 recv 将不能成功。通常需要多次调用函数，直至成功读取数据。

但是这样做会导致程序处于忙等状态，使得执行效率太低而严重影响程序性能。

实验指导

本实验服务器端使用非阻塞模式的套接字，因此，在一个连接完成后，客户还未发信息时，仍可以返回执行 accep()函数接收下一个连接请求，从而可以做到同时与多个客户保持连接。即在一个线程中可以同时为多个客户端提供服务。

由于服务器同时要与多个客户保持连接，因此需要保存多个已连接套接字描述符，为便于操作，需要定义一个 int 类型的数组来保存它们。该数组按如下方式定义：

```
#define NUMSERVER 10
    ……
SOCKET sock[NUMSERVER];
```

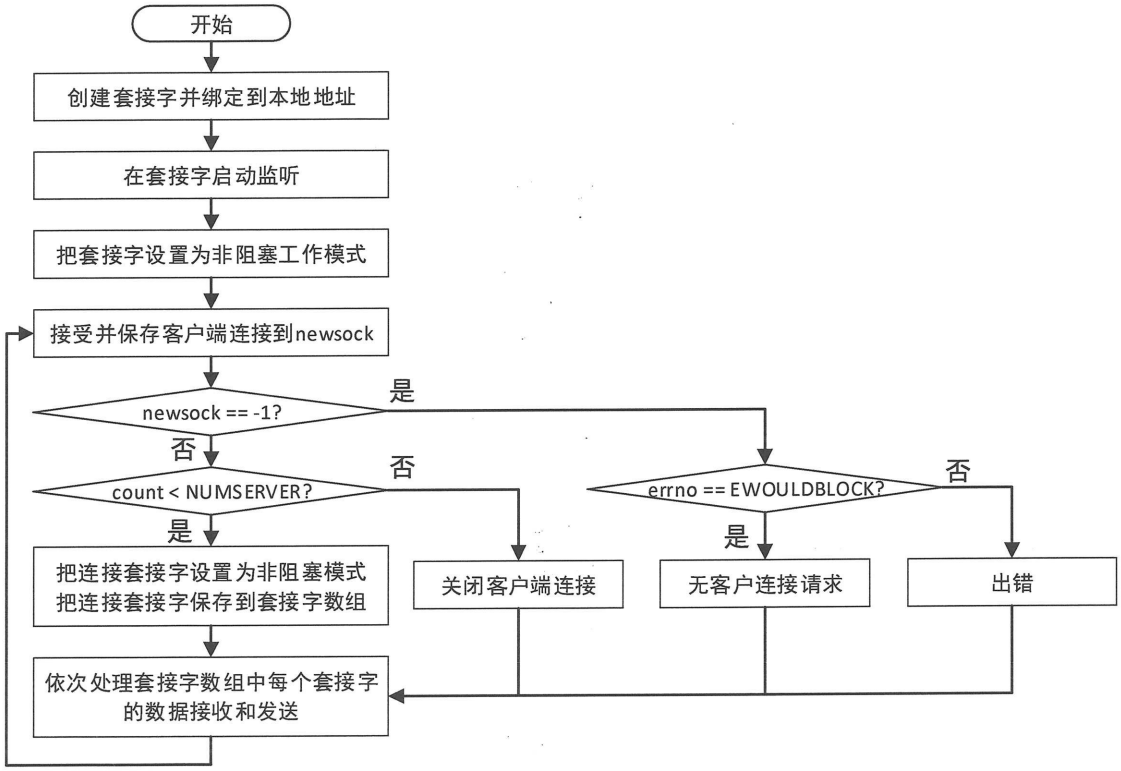
数组中只保存尚未关闭的套接字描述符，已关闭的套接字描述符应从数组中删除；数组中所有的已连接套接字描述符始终保持在从下标 0 开始的元素中连续存放；每当 accept()函数成功返回一个新的已连接套接字的描述符时，该套接字都保存在数组中其它套接字描述符之后。

为达到上述目的，首先需要定义一个变量 count，该变量用于保存数组中已存入的已连接套接字个数，即：count 保存了当前已建立连接的客户端数量。新的连接套接字应保存在下标为 count 的数组元素中；其次，为了保证所有未关闭的已连接套接字在数组中连续存放，每关闭一个已连接套接字，在数组中其描述符后的每个套接字描述符都应前移。

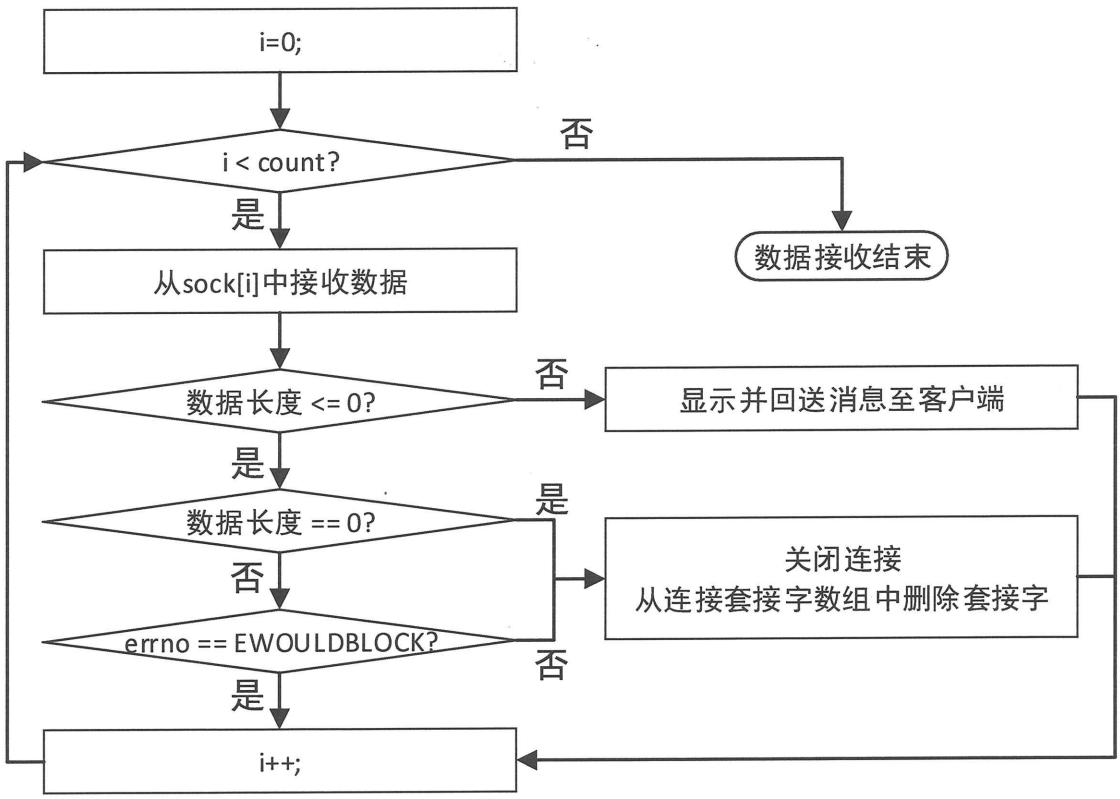
数组中的第 i 个套接字被关闭时，可使用如下代码将其后套接字描述符前移：

```
//关闭第i个套接字closesocket( newsock[i] );
for(int j=i;k<count-1;j++)
    newsock[j]=newsock[j+1];
count--; //数组中套接字总数减1
```

依据上述原理，设计主程序的架构如下：



把上面流程图中处理套接字数组中每个连接套接字上数据接收/发送过程细化为如下程序流程图。



实现程序的源代码示例如下:

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <errno.h>
4  #include <unistd.h>
5  #include <string.h>
6  #include <fcntl.h>
7  #include <sys/types.h>
8  #include <sys/socket.h>
9  #include <arpa/inet.h>
10
11 #define PORT 20000
12 #define BACKLOG 5
13 #define BUFLEN 256
14 #define NUMSERVER 10
15 void main(void)
16 {
17      int lstsock, newsock;
18      int sock[NUMSERVER];
19      int count=0;
20      int flags;
21      int i;
22      int len;
23      int addrlen;
24      struct sockaddr_in saddr;
25      struct sockaddr_in caddr;
26      struct sockaddr_in paddr;
27      char buf [BUFLEN];
28      char msg[] = "Connect success. Please send a message to me.";
29
30      addrlen = sizeof (saddr);
31      if ((lstsock = socket (PF_INET, SOCK_STREAM,0)) == -1)
32      {
33           fprintf (stderr, "Create socket error: %s\n", strerror (errno));
34           exit (1);
35      }
36      memset (&saddr, 0, addrlen);
37      saddr.sin_family = AF_INET;
38      saddr.sin_port = htons (PORT);
39      saddr.sin_addr.s_addr = htonl (INADDR_ANY);
40      if (bind (lstsock, (struct sockaddr*) &saddr, addrlen) == -1)
41      {
42           fprintf (stderr, "Bind socket error: %s\n", strerror(errno));
43           exit (1);
44      }
```

```
45      listen (lstsock, BACKLOG);
46      if((flags = fcntl(lstsock, F_GETFL, 0)) <0 )
47      {
48          fprintf (stderr, "Error: %s\n", strerror(errno));
49          exit (1);
50      }
51      flags |= O_NONBLOCK;
52      if(fcntl(lstsock, F_SETFL, flags) <0)
53      {
54          fprintf (stderr, "Error: %s\n", strerror(errno));
55          exit (1);
56      }
57      while (1)
58      {
59          addrlen = sizeof(caddr);
60          newsock = accept (lstsock, (struct sockaddr*) &caddr, &addrlen);
61          if (newsock == -1)
62          {
63              if(errno != EWOULDBLOCK)
64              {
65                  fprintf (stderr, "Accept client connection error: %s\n", strerror(errno));
66              }
67          } else if( count < NUMSERVER)
68          {
69              flags = fcntl(newsock, F_GETFL, 0);
70              flags |= O_NONBLOCK;
71              fcntl(newsock, F_SETFL, flags);
72              sock[count++] = newsock;
73              printf ("Got connection from: %s: %d\n", inet_ntoa (caddr.sin_addr), ntohs
(caddr.sin_port));
74              send (newsock, msg, sizeof(msg), 0);
75          } else {
76              close (newsock);
77          }
78          for(i=0; i < count; i++)
79          {
80              memset (buf, 0, sizeof(buf));
81              if((len=recv (sock[i], buf, sizeof(buf), 0))<=0)
82              {
83                  if(errno != EWOULDBLOCK || len == 0)
84                  {
85                      addrlen = sizeof(paddr);
86                      memset(&paddr,0,sizeof(paddr));
87                      getpeername(sock[i],(struct sockaddr*)&paddr,&addrlen);
```

```
88                          printf ("%s:%d Closed\n",  inet_ntoa (paddr.sin_addr), ntohs (paddr.sin_port));
89                          close(sock[i]);
90                          for(int j=i;j<count-1;j++)
91                              sock[j] = sock[j+1];
92                          count --;
93                      }
94                  }
95                  else
96                  {
97                      addrlen = sizeof(paddr);
98                      memset(&paddr,0,sizeof(paddr));
99                      getpeername(sock[i],(struct sockaddr*)&paddr,&addrlen);
100                         printf ("Got client message: %s", buf);
101                         printf ("\t from %s:%d \n", inet_ntoa (paddr.sin_addr), ntohs (paddr.sin_port));
102                         send(sock[i],buf,strlen(buf),0);
103                         if (!strcmp (buf, "bye"))
104                         {
105                             printf ("Stop Server! \n");
106                             close (lstsock);
107                             for(i=0;i<count;i++)
108                                 close(sock[i]);
109                             exit(0);
110                         }
111
112                     }
113                 }
114             }
115     }
```